



sellsell marketplace updates:

SHAZ EJ



consolidated status as of **20 June 2026 – 2:17 AM.**

SellSell / PickPic Progress Report

Completed Since Last Review

1. P0 Silent Logout Root Cause Identified

Status: Root cause found

Problem:

- Users were being logged out after page refreshes or route changes.

Root Cause:

- Multiple `AuthProvider` instances were mounted in:
 - `(public)`
 - `(admin)`
 - `(account)`
 - `(support)`
 - `(vision)`
- Each provider was independently calling `/api/auth/refresh`.
- Refresh token rotation caused one request to succeed and another to fail with:
 - `REFRESH_TOKEN_REVOKED`
- Failed provider cleared user state and triggered logout.

Fix Implemented:

- Removed nested `AuthProvider` wrappers.
- Root `src/app/layout.tsx` remains the single authentication provider.

Expected Result:

- Only one refresh request.
- Stable session persistence.
- No unexpected logout.

2. P0 Admin Dashboard Crash Root Cause Identified

Status: Fixed

Problem:

- `/admin` crashed with React Error #419.

Root Cause:

6/18/26 5:55:21 PM



- `FormLabel` was used outside a `FormProvider`.
- `useFormContext()` returned `null`.
- **Crash occurred when:**
- `const { getFieldState } = useFormContext()`

Fix Implemented:

- **Replaced:**
 - `FormLabel`
 - `FormItem`

with:

- `Label`
- regular div wrappers

Expected Result:

- Admin dashboard loads successfully.
- Search filter section no longer crashes.

3. Explore Page Recommendations Crash

Status: Fixed

Problem:

`items.map` is not a function

Root Cause:

Recommendations API returns:

```
{
  "success": true,
  "data": {
    "products": [...]
  }
}
```

Frontend expected:

```
{
  "data": [...]
}
```

Fix Implemented:

- Updated `parseListings()`
- Added support for:
 - `raw.data.products`



- o `raw.data.data`
 - o `raw.results`
 - o `raw.listings`
- Added `Array.isArray()` safety checks.

Expected Result:

- Recommendations section renders correctly.
- No more `.map()` crash.

4. TypeScript Build Issues Fixed

Status: Fixed

Additional fixes were made:

Chat Route

Fixed:

Object is possibly undefined

Auth Refresh Tests

Fixed:

Vitest hoisting error

using:

```
vi.hoisted()
```

5. Validation Completed

Reported results:

TypeScript

```
npm run typecheck
```

Passed

Lint

```
npx next lint
```

6/18/26 5:55:21 PM

Passed

Tests

```
npx vitest run --exclude tests/e2e/
```

Passed

Reported:

- 5 test suites
- 32 tests

All successful.

6. Additional Admin Investigation

You also found another possible admin issue:

Radix Select components were using:

```
value=""
```

and later changed to:

```
value="all"
```

to avoid select-state crashes.

This appears to be a secondary stabilization fix.

Current Project Status

Core Marketplace

Area	Status
Authentication	✓
Buyer AI Search	✓
Seller AI Assistant	✓
Demand Capture Agent	✓



Area	Status
Moderation Pipeline	✓
Embeddings + Qdrant	✓
Recommendations	✓
Chat Interface	✓
Seller Dashboard	✓
Admin Dashboard	✓
Explore Page	✓

Remaining Major Work

Notifications

Current:

- Infrastructure exists

Missing:

- Email delivery
- Push notifications
- User preferences

Estimate:

1-2 days

Admin Dashboard Completion

Current:

- No longer crashing

Still Needed:

- Moderation review tools
- Demand analytics
- Search analytics
- User analytics

Estimate:

2-3 days



Mobile App

Current:

- Partial implementation

Still Needed:

- Seller listing creation
- Listing management
- Mobile polish

Estimate:

3-5 days

QA & Production Hardening

Still Needed:

- Load testing
- Security testing
- Stress testing
- Production monitoring

Estimate:

2-3 days

My Updated Assessment

Before these fixes:

~85-90%

After these fixes:

~93-95%

The biggest blockers found during QA:

1. Silent logout
2. Admin crash
3. Explore crash

have now been addressed according to the implementation reports.

The next thing I would have Antigravity do is:



Priority 1

Run full production QA again against:

- Login
- Refresh
- Explore
- Admin
- Seller flow
- Demand Capture

to verify the fixes actually work in browser.

Priority 2

Finish Admin Dashboard features.

Priority 3

Implement Notifications.

Priority 4

Complete Mobile App seller flow.

Send me the next report and I'll continue maintaining the project timeline and update the true completion percentage.



Based on where the project stands today, I would do these **one by one in this exact order**.

Step 1 — Verify the 3 Critical Fixes on Production

Before building anything else, confirm that the fixes actually work on the live system.

Test

1. Login
2. Refresh page
3. Go to Explore
4. Return to Chat
5. Go to Admin
6. Go to Seller Dashboard
7. Logout/Login again

Expected

- No silent logout
- Admin loads
- Explore recommendations load
- No console errors

Goal: Close the P0 bugs permanently.

Step 2 — Complete Admin Dashboard

This is the biggest missing business tool.

Build

Moderation Queue

Show:

- Pending listings
- Approved listings
- Rejected listings

Actions:

- Approve
- Reject



- Retry moderation

Demand Capture Dashboard

Show:

Toyota Prado → 50 requests

iPhone 17 → 40 requests

Mercedes S400 → 25 requests

This becomes extremely valuable for Fahad.

Goal: Fahad can see what buyers want.

Step 3 — Notifications System

Connect:

Email

- New demand match
- New message
- Listing approved

Push Notifications

- Mobile
- Browser

Start with email first.

Goal: Users come back automatically.

Step 4 — Seller Mobile Flow

Currently one of the most important commercial features.

Verify:

- Login



- Create listing
- Upload photos
- Edit listing
- Delete listing
- View messages

on iPhone and Android.

Goal: Seller can operate entirely from mobile.

Step 5 — End-to-End Production QA

Run complete QA:

Buyer

Search:

- Cars
- Real estate
- Electronics
- Arabic
- English

Seller

Create:

- With image
- Without image

Demand Capture

Search nonexistent products.

Admin

Review listings.

Goal: No major bugs remain.

Step 6 — Hot Demand Dashboard



This is the first "wow" feature for business users.

Show:

Product	Searches
Toyota Prado	120
iPhone 17	80
Mercedes S400	45

This tells sellers what to import and sell.

Goal: Generate marketplace intelligence.

Step 7 — Smart Matching Engine

When a seller adds:

Toyota Prado 2024

Automatically:

- Find interested buyers
- Notify them
- Open conversation

This creates liquidity.

Goal: Faster transactions.

Step 8 — Analytics Dashboard

Show:

- Top searches
- Top categories
- Top cities
- Conversion rate
- Listings created
- Listings sold

Goal: Management visibility.



Step 9 — Production Monitoring

Add:

- Sentry
- Error tracking
- API monitoring
- Queue monitoring
- Ollama monitoring
- RunPod health checks

Goal: Detect problems before users do.

Step 10 — Launch Preparation

Before launch:

- Privacy Policy
- Terms of Use
- Contact page
- Notification settings
- Backup strategy
- Recovery plan

Goal: Commercial readiness.

If I were managing the project today, I would tell Antigravity:

Today: Step 1 (verify fixes)

Tomorrow: Step 2 (Admin Dashboard)

Day 3: Step 3 (Notifications)

Day 4-5: Step 4 (Mobile seller flow)

Day 6: Step 5 (Full QA)

Day 7: Production release candidate

That gets SellSell very close to a public MVP launch.